

Panduan Technical Defense

Aplikasi MCBT - Computer-Based Test

Rangkuman keputusan teknis & alasan di baliknya - siap dibuktikan langsung di kode

1. Kenapa memilih domain CBT / Ujian?

Ujian berbasis kertas punya banyak titik lambat dan rawan salah: cetak soal, distribusi ke ruangan, koreksi manual, lalu rekap nilai manual. Domain ini juga punya aturan bisnis nyata yang bisa dibuktikan di kode - KKM, bobot soal, negatif marking, attempt terbatas, jendela waktu ujian, ranking - sehingga bukan sekadar CRUD kosong.

- Digitalisasi ujian adalah kebutuhan nyata sekolah di Indonesia (istilah domain memakai NIS, KKM, tahun ajaran ganjil/genap, wali kelas).

2. Siapa target user?

- Admin sekolah: kelola master data (siswa, guru, mapel, tahun ajaran), akun, laporan soal.
- Guru: buat bank soal & ujian, review soal sebelum diujikan, nilai esai, tangani laporan soal.
- Siswa: mengerjakan ujian dengan mekanisme anti-kecurangan, lihat hasil & pembahasan, laporkan soal bermasalah.

Bukti: tiga role ini dipetakan ke grup route berbeda (router.go) dan menu sidebar berbeda (AppShell.vue).

3. Business problem yang diselesaikan

- Kecurangan: timer server-authoritative, token masuk ujian, randomisasi soal/opsi per attempt, satu attempt aktif per siswa.
- Koreksi manual: skoring otomatis PG/B-S/Multi/Isian (gradeObjective), esai lewat lembar grading khusus, publish hasil sekali klik.
- Soal bocor/tidak valid: fitur laporkan soal oleh siswa + halaman review soal sebelum ujian (/exams/:id/review).
- Rekap & ranking manual: otomatis per ujian/kelas + ekspor Excel/PDF.
- Kolaborasi guru: data-scope - guru kelola bank/ujian miliknya, bisa melihat bank kolega (read-only).

4. Kenapa memilih arsitektur ini?

Modular monolith berlapis: handler -> usecase -> repository. Handler hanya parsing HTTP & response, usecase berisi aturan bisnis, repository hanya akses data.

- Mudah dites: 24 usecase difactor ke consumer-defined interface (interfaces.go) sehingga 94 unit test jalan tanpa DB (fakes_test.go).
- Test menemukan bug nyata: nil-pointer di questionInExam saat ujian tanpa bank soal - tertangkap sebelum produksi.
- Monolith dipilih karena skala sekolah bukan ratusan ribu RPS: deployment sederhana (satu docker-compose), transaksi lintas modul memakai ACID PostgreSQL.

5. Kenapa authentication menggunakan cookie?

- XSS: token di localStorage bisa dibaca JS yang disuntik; HttpOnly cookie mustahil dibaca JS. Aplikasi ini kaya rich-text (editor soal) sehingga permukaan XSS lebih besar.
- Cookie otomatis terkirim setiap request - FE tidak mengelola token manual.

Trade-off-nya adalah CSRF, dan itu ditangani berlapis (lihat poin 7).

6. Fungsi HttpOnly / Secure / SameSite

- HttpOnly: cookie tidak bisa diakses `document.cookie` -> token aman dari pencurian via XSS.
 - Secure: cookie hanya dikirim lewat HTTPS -> anti penyadapan jaringan (dev false, produksi true via env `COOKIE_SECURE`).
 - SameSite=Strict: cookie tidak dikirim pada request lintas situs -> lapisan pertama anti-CSRF.
- Semua dikonfigurasi via env - tunjuk `config.go` dan `cookies.go` saat menjelaskan.*

7. Cara mengatasi CSRF

Defense in depth, tiga lapis:

- 1. SameSite=Strict sebagai lapisan pasif - cookie tak ikut request lintas situs.
- 2. Double-submit cookie: server menanam cookie `csrf_token`; FE (interceptor axios) wajib mengirim header `X-CSRF-Token` bernilai sama untuk POST/PUT/PATCH/DELETE; middleware `CSRFProtection` membandingkan keduanya. Attacker domain lain tidak bisa membaca cookie -> tidak bisa menyusun header.
- 3. Bukti nyata: dua bug dulu (lapor soal & tangani laporan memakai fetch tanpa header) ditolak 403 oleh middleware, lalu diperbaiki lewat instance axios.

8. Cara refresh authentication

- Dua cookie terpisah: `access_token` (15 menit) dan `refresh_token` (7 hari).
- Interceptor FE menangkap 401 -> panggil POST `/auth/refresh-token` -> server validasi refresh JWT -> terbitkan access baru -> request asli di-replay. User tidak pernah logout paksa.
- Rotasi: refresh juga menerbitkan pasangan token baru.
- Revocation: `token_version` di DB; reset password menaikkan versi sehingga semua token lama otomatis ditolak (Session revoked). Dites di `TestRefresh_StaleTokenVersion`.

9. Cara authorization diterapkan

- Role-based (`RequireRoles`): grup route `adminOnly`, `staffOnly` (admin+guru), `candidate` (siswa) di `router.go`.
- Data-scope/ownership: bank soal & ujian punya `created_by`; middleware `dataScopeGuard` + `AccessUsecase.Assert*Owner` memverifikasi berantai (laporan -> attempt -> ujian -> `created_by`).
- Response membawa flag `can_manage` sehingga FE menyembunyikan tombol edit/hapus data orang lain.
- Siswa hanya melihat ujian yang di-assign kepadanya dan hasil miliknya sendiri.

10. Kenapa menggunakan Composite (aggregated) API

- Satu endpoint mengembalikan objek gabungan siap pakai: GET `/exams` menyertakan `subject`, `question_bank`, `academic_year`, `attempts_count`; dashboard mengagregasi 8 metrik dalam satu call; review soal mengembalikan soal+opsi+kunci+pembahasan per section.
- Alasan: mengurangi round-trip dan state di FE (satu halaman ujian = satu request), format envelope konsisten `{success, message, data}`.
- Trade-off: over-fetching untuk kasus tertentu - diterima karena konsumennya jelas (SPA sendiri), dan N+1 dicegah di SQL (subquery `attempts_count`, `Preload`).

11. Cara menangani concurrent request

- Jawaban siswa: unique index (`attempt_id`, `question_id`) + UPSERT -> request simultan tidak menciptakan duplikat.

- Submit idempotent: attempt sudah submitted -> return tanpa menulis ulang (dites di TestAttemptSubmit/idempotent).
- Operasi multi-baris (clone bank, finalize) dibungkus transaksi database.
- Sumber waktu = server: heartbeat menghitung sisa waktu dari jam server (now bisa diinjeksi & dites), jam browser tidak dipercaya.
- Satu attempt aktif per (exam, student) dijaga FindActive + constraint; connection pool Postgres dikonfigurasi (DB_MAX_OPEN_CONNS).

12. Desain database jika data menjadi sangat besar

Baris terbesar adalah exam_attempts + exam_answers (tumbuh linear per ujian). Rencananya:

- Partitioning native Postgres per exam_id atau per bulan started_at - query selalu ter-filter exam/tanggal.
- Indexing sudah disiapkan sejak migrasi: composite (exam_id, student_id), idx_attempts_student, unique (attempt_id, question_id), index created_by.
- Read replica untuk beban laporan/ekspor; arsip ujian lama ke tabel arsip.
- Media keluar dari DB ke MinIO/S3 (hanya media_key di DB); Redis (sudah ada di compose) untuk cache ranking/dashboard.

13. Kenapa menggunakan queue?

Jawaban jujur: belum terpakai saat ini, Redis sudah disiapkan di compose - keputusan sadar (YAGNI). Queue dibutuhkan untuk pekerjaan berat fire-and-forget, kandidat nyatanya:

- Penilaian massal saat ratusan siswa submit serentak (calculate-grades jadi job).
- Generate ekspor PDF/Excel besar agar tidak memblok HTTP request.
- Notifikasi (hasil rilis / pengumuman ujian). Implementasi pilihan: async/river di atas Redis yang sudah ada.

14. Cara menangani failed job

- Retry dengan exponential backoff untuk error transient (DB timeout, S3 5xx).
- Idempotency: job penilaian aman diulang karena UpdateGrading/UpdateAttemptScore berbasis ID dan total dihitung ulang dari DB, bukan akumulasi.
- Dead-letter queue setelah N kali gagal + log terstruktur (slog) dengan request_id untuk investigasi.
- Partial failure: CalculateGrades memproses per attempt; esai tanpa nilai dilewati, bukan error.

15. Cara memastikan file upload aman

- Ukuran dibatasi keras: MaxUploadMB dicek di usecase + client_max_body_size di Nginx + MaxMultipartMemory Gin.
- Nama file asli dibuang -> disimpan dengan random key (migrasi media_key_path) -> tak ada path traversal / overwrite.
- Disimpan di MinIO/S3, bukan disk aplikasi -> terpisah dari runtime, tidak bisa dieksekusi.
- Disajikan lewat endpoint berotorisasi GET /media/:id/file dengan content-type terkontrol; hanya untuk gambar soal via editor rich-text oleh staff.

16. Cara meningkatkan performance aplikasi

Sudah dilakukan:

- Pagination di semua list + meta; anti-N+1 (Preload terarah, attempts_count via subquery, agregat dashboard dihitung 1 query SQL).
- Index strategis (composite & unique) sejak migrasi; SPA statik dilayani Nginx (cache + gzip), API hanya JSON ringan; skor dihitung di SQL (SUM).

Roadmap (dengan alasan):

- Cache Redis untuk ranking/dashboard (stack sudah ada), CDN di depan MinIO untuk media, partitioning sesuai poin 12.

17. Trade-off dari arsitektur yang dipilih

- Modular monolith: deploy mudah + transaksi ACID lintas modul VS scale-out per-modul terbatas; dimitigasi dengan batas lapisan yang disiplin.
- JWT di cookie HttpOnly: anti-XSS VS rentan CSRF -> dibayar pajaknya dengan CSRF token + SameSite.
- Server-side timer: anti-cheat kuat VS butuh heartbeat berkala; mitigasi autosave agar jawaban tidak hilang.
- Composite response: FE sederhana VS over-fetch; konsumen jelas (SPA sendiri) sehingga terkendali.
- GORM: produktivitas VS kontrol query - query kompleks tetap raw SQL (dashboard, ranking): pendekatan hybrid.
- Interface + hand-written fakes: test cepat tanpa DB & tanpa dependensi mock VS fakes harus sinkron dengan kontrak repo ((nil,nil) vs ErrRecordNotFound).
- Redis disiapkan tanpa queue: tidak over-engineer VS fitur async menunggu kebutuhan nyata.

18. Kunci saat presentasi

Gunakan pola: keputusan -> alasan -> bukti di kode (sebut file) -> trade-off yang disadari -> mitigasinya.

Contoh kalimat pembuka: "Saya tidak memilih teknologi karena populer, tapi karena masalah yang diselesaikan

- dan setiap trade-off-nya saya akui sekaligus saya mitigasi."

Angka pendukung: 94 unit test usecase (coverage in-scope penuh), 5 paket Go + 7 component test FE hijau, 21 migrasi terstruktur, docker-compose 5 layanan.